

[OPTIONAL] Prometheus Metrics Reporting with Grafana Visualization

A metrics-native alternative to three of the seven Flink reports: emit them as Micrometer meters from the producer interceptor and let Prometheus do the windowing at read time. **Additive and opt-in** — it does not replace the Flink reports. The companion one-command showcase (Prometheus + Grafana on minikube) has its own runbook: k8s/monitoring/README.md.

Table of Contents

- [1.0 Why three of seven](#)
 - [2.0 Enabling the exporter](#)
 - [3.0 The three reports as PromQL](#)
 - [4.0 Consume-side meters](#)
 - [5.0 Operational queries \(cookbook\)](#)
 - [6.0 Mapping questions to PromQL](#)
 - [6.1 Easy — single record, single trace](#)
 - [6.2 Easy → Medium — single per-minute aggregates](#)
 - [6.3 Medium — cross-window deltas, anomalies, multi-report joins](#)
 - [6.4 Hard — tail latency, drift, correlation](#)
 - [6.5 Harder — forensic replay, compliance, cross-system](#)
 - [7.0 What stays in Flink — and two deliberate gaps](#)
 - [8.0 One-command showcase: Prometheus + Grafana on minikube](#)
-

1.0 Why three of seven

Is Flink overkill for these reports? **Yes and no** — it's not all about stateless aggregation. Of the seven reports, **three are pure stateless scalar aggregation** keyed on bounded-cardinality dimensions (service / topic / hop_count — never `trace_id`):

- **latency_1m** (`10_latency_report.fql`) — avg/max latency per (pipeline, origin_service, this_topic).
- **topology_1m** (`20_topology_report.fql`) — produce-edge record counts.
- **hop_distribution_1m** (`30_hop_distribution.fql`) — records per hop_count.

These don't need a stream processor. The **producer interceptor** already has every value in scope on each `send()`, so it can emit them as **Micrometer** meters and let **Prometheus** do the 1-minute windowing at query time (`rate()` / `increase()`), with **Grafana** on top. The other four reports — **latency_percentiles** (mergeable T-Digest), **coverage**, **bipartite_topology**, **stuck_trace** — are per-`trace_id` *stateful* or *absence-of-event* problems Prometheus can't express, so they **stay in Flink**. This path is **additive and opt-in** (off by default); it doesn't replace the Flink reports — it's the cheaper way to serve the three that are metrics, not stream processing.

Emission lives in one place — `PrometheusIsotopeMetrics.java` — mirroring how **TDigests** centralizes the sketch contract.

Packaged as a library. The tracing is an adoptable, publishable library, split so metrics are optional: `kafka-isotope-core` carries the propagation (interceptor, headers, consume markers — Jackson + Kafka only), and `kafka-isotope-metrics` adds the Micrometer/Prometheus exporter. The core routes emissions through a no-op `IsotopeMetricsSink` until `kafka-isotope-metrics` registers the Prometheus one, so propagation pulls no metrics dependency. The runnable stages below (`:app`) are the reference consumer — see [kafka-isotope-core/README.md](#) to adopt it elsewhere.

2.0 Enabling the exporter

Pass `-Dmetrics.prometheus.enabled=true` to any long-running stage. **Producing** stages (`hop` / `enrich` / `fulfill`) emit the produce-side meters via the interceptor; **consuming** stages emit the consume-side meters via the marker path ([Consume-side meters](#)) — `hop` does both (it consumes *and* produces), and the terminal `consume` / `ship` stages emit the consume side alone. The app serves the meters at `GET /metrics` on `metrics.prometheus.port` (default `9404`) via the JDK's built-in HTTP server — no extra runtime.

```
# Prereq: 'make kafka-pf-up' is up. Run the enrich stage with metrics
exposed:
./gradlew :app:run -Dmetrics.prometheus.enabled=true --args="enrich"
# → metrics on http://localhost:9404/metrics

# In another shell, drive traffic so the meters move (origin produce):
for i in {1..30}; do ./gradlew :app:run --args="place burst-$i" -q; sleep 5;
done

# Scrape:
curl -s localhost:9404/metrics | grep isotope_
```

Point a Prometheus scrape job at each stage's `/metrics` and the three reports become PromQL queries (no report topics, no Control Center — Grafana instead). Unlike the Flink jobs, there's **no watermark wait**: cumulative counters update on every send and Prometheus windows them at read time.

Running the Grafana showcase? Add `-Dmetrics.prometheus.port`. The command above binds the default `9404`, which is right for a single stage you scrape by hand — but stages share one host, so they cannot all bind it, and §8.0's Prometheus config scrapes a fixed port per stage: `enrich`→`9410`, `fulfill`→`9411`, `ship`→`9412`. Omit the flag and the stage still exports happily on `9404` while every Prometheus target reads `connection refused` and Grafana shows no data. Check with `kubectl port-forward -n monitoring svc/prometheus 9090:9090` → Status → Targets.

Reading the latency metric — mind the backlog. `hop` / `consume` use a *random* consumer group with `auto.offset.reset=earliest`, so a fresh stage **replays the whole topic from offset 0**. Latency is `now - origin_ts`, so days-old replayed records report enormous values — e.g. an `isotope_hop_latency_seconds_sum` of ~9,000,000 over 58 records (~43 h average) is just the backlog, not steady-state latency (the Flink `latency_1m` report shows the same, by the same definition). The tell: `_count` exceeds the number of records you just sent. Two ways to see realistic latency: drain the backlog and read `rate(isotope_hop_latency_seconds_sum[1m]) / rate(..._count[1m])` (the windowed rate ignores the stale backlog once it stops growing), or skip

the backlog entirely with `-Disotope.consume.from=latest` so the stage only consumes records produced after it starts:

```
./gradlew :app:run -Dmetrics.prometheus.enabled=true -
Disotope.consume.from=latest --args="enrich"
```

3.0 The three reports as PromQL

Two meters cover all three reports — a `Timer` whose count doubles as the topology edge count, plus a `Counter` for the hop histogram:

Report	Meter (Prometheus name)	Labels
latency_1m + topology_1m	isotope_hop_latency_seconds_{count,sum,max} (Timer)	pipeline, origin_service, this_service, this_topic
hop_distribution_1m	isotope_hop_records_total (Counter)	pipeline, this_topic, hop_count

```
# latency_1m – avg latency (seconds)
sum by (pipeline, origin_service, this_topic)
(rate(isotope_hop_latency_seconds_sum[1m]))
/ sum by (pipeline, origin_service, this_topic)
(rate(isotope_hop_latency_seconds_count[1m]))

# latency_1m – max
max by (pipeline, origin_service, this_topic)
(isotope_hop_latency_seconds_max)

# topology_1m – records per produce edge
sum by (pipeline, origin_service, this_service, this_topic)
(increase(isotope_hop_latency_seconds_count[1m]))

# hop_distribution_1m – records per hop_count
sum by (pipeline, this_topic, hop_count)
(increase(isotope_hop_records_total[1m]))
```

4.0 Consume-side meters

The three meters above all come from the **produce** side — the interceptor on `send()`. The **consume** side adds three more. Two — the edge counter and the time-to-consume latency — are emitted by `IsotopeContext.recordConsume` right beside the consume-edge marker it writes to `isotope_consume_edge_markers`. The third, `isotope.consume.age`, is emitted **once per consumed record on whichever path the consumer takes**: continuing consumers report it from the **adoption path** (`IsotopeContext.adoptFromRecord(record, service)`), and terminal consumers — which never

adopt — report it from the **marker path** (`recordConsume`, guarded on `current() == null` so a stage that does both, like `hop`, never double-counts). So age fires on every consuming stage: `hop` via adoption, the terminal `consume` / `ship` stages via the marker. Same gating — no-op unless the exporter is started.

Signal	Meter (Prometheus name)	Labels
consume-edge counts (topic→consumer)	<code>isotope_consume_records_total</code> (Counter)	<code>pipeline</code> , <code>this_topic</code> , <code>consumer_service</code>
time-to-consume latency (origin→consume)	<code>isotope_consume_latency_seconds_{count,sum,max}</code> (Timer)	<code>pipeline</code> , <code>origin_service</code> , <code>consumer_service</code> , <code>this_topic</code>
origin→adoption age (how stale at pickup)	<code>isotope_consume_age_seconds_{count,sum,max}</code> (Timer)	<code>pipeline</code> , <code>origin_service</code> , <code>consumer_service</code> , <code>this_topic</code>

```
# consume edges — records consumed per (topic → consumer) per minute
sum by (pipeline, this_topic, consumer_service)
(increase(isotope_consume_records_total[1m]))

# time-to-consume — avg seconds from trace origin to consumption
sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_latency_seconds_sum[1m]))
/ sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_latency_seconds_count[1m]))

# consume age — avg seconds a record had aged by the time a service adopted
it
sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_age_seconds_sum[1m]))
/ sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_age_seconds_count[1m]))
```

These are **net-new signals**, not a port of a Flink report. `isotope_consume_records_total` is the *consume half* of the bipartite topology — the topic→consumer edge tallies — and `isotope_consume_latency_seconds_*` is an origin→consume figure no Flink report computes (the Flink `latency_1m` measures origin→**produce**-hop). The **full bipartite topology** report still stays in Flink: stitching produce and consume edges per `trace_id` into one graph is per-trace stateful, which a counter can't do. What you get here is the bounded-cardinality edge *counts* and a time-to-consume distribution — cheap, windowable at read time, and free of the watermark wait.

`isotope_consume_age_seconds_*` measures the **same now – origin_ts** quantity as `isotope_consume_latency_seconds_*`, but it's the **universal consume-side age signal** — emitted once on every consuming stage, where latency only fires on the marker path. That's the point of having both: query `isotope_consume_age_*` to get consume-lag across **all** consumers (including `hop`-style stages that adopt-and-forward without ever writing a marker), and the timer's `_count` doubles as the per-

(`consumer_service`, `topic`) consume rate. At a terminal consumer the age equals that consumer's latency by definition (same `now - origin_ts`); the two diverge in coverage, not value — latency is marker-only, age is everywhere.

The same **backlog caveat** from [Enabling the exporter](#) applies: `hop` / `consume` / `ship` use a random consumer group with `auto.offset.reset=earliest`, so a fresh stage replays from offset 0 and both `isotope_consume_latency_seconds_sum` and `isotope_consume_age_seconds_sum` then reflect days-old origins, not steady-state. Read the windowed `rate(...sum[1m]) / rate(...count[1m])`, or start with `-Disotope.consume.from=latest`.

5.0 Operational queries (cookbook)

The queries above mirror the three reports. These are the **operational** angles you'd reach for when running the demo or alerting on it — they lean on the `status` label (present on every meter: `success` on the happy path) and on read-time ratios. All are `pipeline`-scoped via `{pipeline=~"$pipeline"}` when used in the provisioned dashboard.

```
# Hop failure ratio per service (0-1) - clamp the denominator so a quiet
# stage reads 0, not NaN. Grafana: format as "percent (0.0-1.0)".
sum by (this_service) (rate(isotope_hop_records_total{status!="success"}
[5m]))
/ clamp_min(sum by (this_service) (rate(isotope_hop_records_total[5m])), 1e-
9)

# Hop completion - share of traffic reaching the deepest hop (full pipeline).
# hop_count="3" is the terminal depth for the orders.* chain.
sum(rate(isotope_hop_records_total{hop_count="3"}[5m]))
/ clamp_min(sum(rate(isotope_hop_records_total[5m])), 1e-9)

# Consumers reading data older than 30s on average - adoption-lag alert.
( sum by (consumer_service) (rate(isotope_consume_age_seconds_sum[5m]))
/ sum by (consumer_service) (rate(isotope_consume_age_seconds_count[5m])) ) >
30

# Stage liveness - fires for any stage that emitted no record in the last 2m.
sum by (this_service) (increase(isotope_hop_records_total[2m])) == 0

# Scrape health - an exporter is down (host stage not running on its port).
up{job="isotope-stages"} == 0

# Drop-off between adjacent topics - records into enriched vs into fulfilled.
sum(rate(isotope_consume_records_total{this_topic="orders.enriched"}[5m]))
- sum(rate(isotope_consume_records_total{this_topic="orders.fulfilled"}[5m]))
```

Why no `histogram_quantile`? These are Micrometer `Timers` exported as `_sum/_count/_max` — there are **no `bucket series`**, so percentiles aren't available. Use `rate(_sum)/rate(_count)` for the average and `_max` for the recent worst case (a decaying per-step max, not an all-time high). Enable client-side histograms in the exporter if you need true quantiles.

6.0 Mapping questions to PromQL

What can you actually ask these meters? Below, each question carries a verdict:

-  **PromQL** — answerable directly from the meters.
-  **PromQL (approx)** — answerable as a bounded-cardinality *aggregate*; the exact, per-trace form stays in Flink (or the record header).
-  **Header / Flink** — per-`trace_id` or absence-of-event; **no Prometheus equivalent** (these meters never tag `trace_id` — see [PrometheusIsotopeMetrics.java](#)).

6.1 Easy — single record, single trace

Every question here is about **one** record or trace. Prometheus aggregates away identity, so the answer lives on the record's `isotope` header (inspect the record) or in Flink's per-trace reports — **not** in these meters.

Question	Verdict	Where to look
Did my record get tagged?		The <code>isotope</code> header on the record itself — presence = tagged.
What's the origin of this trace?		<code>origin_service</code> / <code>origin_ts</code> fields in the header.
How many hops has <i>this</i> record taken?		The header's hop list length. (The <i>distribution</i> across all records is  — see Easy→Medium.)
Where did this trace go?		Per-trace path = Flink <code>bipartite_topology</code> , or replay the header trail.
Did the trace ID survive a consume-then-produce hop?		App/Flink per-trace; a counter can't follow one <code>trace_id</code> .
Which pipeline does this trace belong to?		Header <code>pipeline</code> field. (<code>pipeline</code> is a meter <i>label</i> , but can't isolate one trace.)

6.2 Easy → Medium — single per-minute aggregates

This is the meters' sweet spot: bounded-cardinality scalar aggregation, windowed at query time.

```
#  End-to-end latency intake → shipping over the last minute
(origin-consume)
  sum(rate(isotope_consume_latency_seconds_sum{consumer_service="shipping-
notification-service"}[1m]))
/ sum(rate(isotope_consume_latency_seconds_count{consumer_service="shipping-
notification-service"}[1m]))

#  The actual service graph – produce edges + consume edges (the edge
*sets*;
#   the full per-trace stitch is Flink's bipartite_topology)
sum by (this_service, this_topic)
(increase(isotope_hop_latency_seconds_count[1m])) # produce side
sum by (this_topic, consumer_service)
(increase(isotope_consume_records_total[1m]))      # consume side

#  Records per topic per minute – proxy for "distinct traces"; a counter
can't
```

```
# COUNT(DISTINCT trace_id), so this counts records, not deduped traces (→ Flink)
sum by (this_topic) (increase(isotope_hop_records_total[1m]))

# ✅ Hop-count distribution – long tails here = retry storms
sum by (hop_count) (increase(isotope_hop_records_total[1m]))

# ✅ Traces hitting the 32-hop ceiling (Isotope.MAX_HOPS). The eviction
*marker*
# itself is an absence-of-event → Flink; the count at the ceiling is
metric-native
sum (increase(isotope_hop_records_total{hop_count="32"}[5m]))

# 🟡 Records each pipeline carries, minute by minute (records, not distinct
traces)
sum by (pipeline) (increase(isotope_hop_records_total[1m]))
```

6.3 Medium — cross-window deltas, anomalies, multi-report joins

Cross-window deltas and "newly appeared" detection are a Prometheus *strength*; per-trace coverage and stuck-trace detection are where it hands off to Flink.

```
# ✅ Did latency get worse after the 2pm deploy? – now vs. before, via
offset
( sum(rate(isotope_hop_latency_seconds_sum[5m])) /
sum(rate(isotope_hop_latency_seconds_count[5m])) )
– ( sum(rate(isotope_hop_latency_seconds_sum[5m] offset 2h)) /
sum(rate(isotope_hop_latency_seconds_count[5m] offset 2h)) )

# 🟡 What % of intake records made it through to the shipping consumer?
# Ratio of rates – NOT a per-trace coverage join (that's Flink's coverage
report)
sum(rate(isotope_consume_records_total{consumer_service="shipping-
notification-service"}[5m]))
/ sum(rate(isotope_hop_records_total{this_topic="orders.placed"}[5m]))

# 🟡 Where are records being dropped? – drop-off between adjacent edges.
# Pinpointing *which* traces dropped = Flink coverage
sum(rate(isotope_hop_records_total{this_topic="orders.enriched"}[5m]))
– sum(rate(isotope_hop_records_total{this_topic="orders.fulfilled"}[5m]))

# ✅ When a new service goes live, when does it first appear in the
topology?
# Series present now but absent an hour ago
group by (this_service) (isotope_hop_latency_seconds_count)
unless
group by (this_service) (isotope_hop_latency_seconds_count offset 1h)
```

Question	Verdict	Why
Are some traces duplicated at the terminal sink?		Dedup is per- <code>trace_id</code> ; a counter can hint (consume > produce) but can't name the dupes → Flink.
Which traces went in but never came out within 60s?		Absence-of-event per trace = Flink <code>stuck_trace_alerts</code> ; Prometheus can't assert a <i>missing</i> series.
Where did each stuck trace last get seen?		Per-trace state → Flink / the header trail.

The pattern: **aggregates and time-deltas → PromQL; identity, dedup, and absence → Flink**. That boundary is the same one [PrometheusIsotopeMetrics.java](#) draws (3 metrics-native reports, 4 stay in Flink), and the next section spells out the two columns that can never cross it.

6.4 Hard — tail latency, drift, correlation

Past this point PromQL is out of the running — not because the questions are exotic, but because every one of them needs either an adaptive sketch or a join across two reports. Listed to complete the [five-tier ladder](#) the root README shows; the "where to look" column is the answer.

Question	Verdict	Where to look
What are p50 / p95 / p99 across the pipeline?		<code>latency_percentiles_flat_1m</code> — a T-Digest sketch inside <code>LATENCY_PERCENTILES</code> , keyed (<code>pipeline</code> , <code>origin_service</code> , <code>this_topic</code>). These Timers publish no <code>_bucket</code> series, so <code>histogram_quantile</code> is unavailable (§5.0, §7.0).
Which edge owns the tail?		Same report — rank <code>p99_ms</code> across <code>this_topic</code> inside one window.
Did the tail get worse after a deploy?		Compare <code>p99_ms</code> for one key across two windows. PromQL's <code>offset</code> trick in §6.3 drifts the average , never the tail.
Is a tail regression global, or confined to one pipeline?		<code>p99_ms</code> per <code>pipeline</code> in the same window; <code>sample_count</code> says whether the key carried enough traffic to trust the number.
Are the slow windows the retry-storm windows?		Join <code>hop_distribution_1m</code> to <code>latency_percentiles_flat_1m</code> on (<code>window_start</code> , <code>pipeline</code> , <code>this_topic</code>) — hop-count outliers against the tail.
Is the average hiding the tail?		<code>avg_latency_ms</code> / <code>max_latency_ms</code> from <code>latency_report_1m</code> against <code>p99_ms</code> on the same key. Prometheus ports avg and max, but not the windowed min (§7.0).

6.5 Harder — forensic replay, compliance, cross-system

Identity, derivation, and absence — the three things a counter structurally cannot hold. Two of these need an opt-in stage, marked inline.

Question	Verdict	Where to look
Reconstruct a full per-trace journey for an audit?		The record's own <code>isotope</code> header hop list, plus <code>bipartite_topology_report_1m</code> for the edges around it and <code>stuck_trace_alerts_1m</code> (<code>last_service</code> , <code>last_topic</code> , <code>last_hop_count</code>) for where it stopped.
Which input records produced this merged output?		Opt-in fan-in provenance (flink-collector.md §2.4): join <code>orders.flink_batched</code> to <code>isotope_merge_edge_markers</code> on <code>merge_trace_id</code> for the whole parent set — <code>contributing_trace_id / _service / _topic / _records</code> , one row per contributing trace per window (the row count equals the merged record's <code>distinct_traces</code> ; <code>SUM(contributing_records)</code> equals its <code>event_count</code>). No aggregate ever forwards a parent's trace, so this is the <i>only</i> way to recover the derivation.
Which stage lost the traces that never reached the terminal consumer?		<code>coverage_report_1m</code> 's <code>distinct_traces</code> , walked topic by topic, locates the drop-off; naming the lost traces takes an anti-join over the <code>isotope</code> view against the consume edges.
Did the isotope survive Flink's own hop?		<code>orders.flink_enriched</code> carries the hop <code>ISOTOPE_APPEND_HOP</code> stamped in-band, so Flink appears in <code>topology_report_1m / bipartite_topology_report_1m</code> as a <code>producer_service</code> like any service (flink-collector.md).
What fraction of records on a topic carried a parseable isotope at all?		Count <code>isotope_raw</code> (unfiltered) against the <code>isotope</code> view, which keeps only rows with <code>x-isotope-trace-id</code> . Prometheus cannot supply the denominator: an untagged record never reaches a meter in the first place.
Which versions produced the current value of this row?		Opt-in , CP-only state-level provenance (root README §3.6): <code>isotope_state_provenance</code> carries one record per emitted state, content-addressed by <code>StateVersion</code> , with the versions it derived from inline in <code>parents</code> . This is the one question the windowed merge collector cannot reach at all — an unbounded <code>GROUP BY</code> , a regular join, and a dedup have no window bounds to key an identity on.
Why was this trace stuck — not just that it was?		Opt-in , CCAF-only AI report (root README §3.3.2): <code>isotope_report_trace_rca_1m.root_cause</code> , one <code>ML_PREDICT</code> call per stuck-trace alert.

7.0 What stays in Flink — and two deliberate gaps

Moving these out isn't free — two columns the Flink reports carry have **no Prometheus equivalent**, and both are documented in [PrometheusIsotopeMetrics.java](#):

- **No `distinct_traces`**. All three SQL reports carry a `COUNT(DISTINCT trace_id)` column. A counter can't dedup, and `trace_id` is unbounded-cardinality so it can't be a label. If you need it, that column stays in Flink (or approximate it with a HyperLogLog sketch).
- **No windowed `min` latency**. A Micrometer `Timer` exposes max but not a per-window min — avg and max port cleanly, min does not.

So the line between "metric" and "Flink job" runs *through* a couple of these reports, not cleanly between them — which is exactly why the move is opt-in rather than a wholesale replacement.

Why `latency_percentiles_1m` is NOT in this list. Percentiles *can* be served by Prometheus — but only via a *classic* histogram (`publishPercentileHistogram()` + `histogram_quantile()`), whose accuracy is **bucket-bound, not adaptive**: error is the width of fixed, pre-chosen buckets, so the tail (p99) is only as good as your bucket layout, and covering a range finely means emitting hundreds of `le` series per tag combo. Prometheus's adaptive answer — *native histograms* (exponential buckets, the closest thing to a `T-Digest`) — isn't emittable through Micrometer yet (experimental, protobuf-only as of late 2024). So the `T-Digest PTF` wins on tail accuracy **and** scales better: its sketch is bounded (~few KB/key) and mergeable, whereas at production volume a Micrometer percentile-histogram's `le`-bucket cardinality grows with the range you need to resolve. The built-in Flink `PERCENTILE` aggregate (exact, pure SQL) is *also* the wrong call at high volume — it retains every value in the window — so percentiles stay a **T-Digest sketch PTF** on purpose (see that file's header for the full rationale). This is a 3-Micrometer / 4-Flink split, not 4/3.

8.0 One-command showcase: Prometheus + Grafana on minikube

To see these meters instead of `curl`-ing `/metrics`, there's an optional, self-contained stack under [k8s/monitoring/](#) — Prometheus + Grafana as minikube pods, with the datasource and a dashboard (all six produce/consume signals above) **auto-provisioned**, so it opens straight to a populated board with no login.

The pipeline stages run on your **host** via `./gradlew :app:run`, not in-cluster — so Prometheus scrapes back across the minikube→host bridge `host.minikube.internal`, one host port per stage (`enrich`→9410, `fulfill`→9411, `ship`→9412; edit [k8s/monitoring/10-prometheus.yaml](#) to change the mapping).

```
make metrics-up          # deploy, wait, port-forward Prometheus+Grafana, open
                           Grafana

# In separate terminals, run each stage on its mapped port (metrics on):
./gradlew :app:run -Dmetrics.prometheus.enabled=true -
Dmetrics.prometheus.port=9410 \
  -Disotope.consume.from=latest --args="enrich"      # fulfill→9411, ship→9412
# ...then drive traffic with `place` so the meters move.

make metrics-down        # stop the port-forwards (pods stay up)
make metrics-delete      # tear the whole showcase down
```

Confirm scraping at Prometheus → [Status](#) → [Targets](#) (a target is `DOWN` until you start its stage — expected). Full runbook and troubleshooting: [k8s/monitoring/README.md](#).